SQL Transactions & Concurrency Exercises - GameVerse Database

Entry Level Transaction Control & Concurrent Access

Duration: 45 minutes
Database: GameVerse (Game Store Database)

Introduction to Transactions

A **transaction** is a sequence of database operations that are treated as a single unit of work. Key principles:

ACID Properties

- **Atomicity**: All operations succeed, or none do (all-or-nothing)
- **Consistency**: Database remains in a valid state
- **Isolation**: Concurrent transactions don't interfere with each other
- **Durability**: Completed transactions are permanent

Transaction Commands

- `BEGIN` or `START TRANSACTION`: Start a transaction
- `COMMIT`: Save all changes permanently
- `ROLLBACK`: Cancel all changes and restore previous state
- `SAVEPOINT`: Create a checkpoint within a transaction

Part 1: Basic Transactions (10 minutes)

Exercise 1.1: Simple Transaction

Write a transaction that:

1. Starts a transaction
2. Inserts a new genre called 'Educational' with description 'Learning-focused games'
3. Commits the transaction

▶ Hint

Exercise 1.2: Transaction with Rollback

Write a transaction that:

1. Starts a transaction
2. Updates the price of game_id 1 to $999.99
3. Rolls back the transaction (cancel the change)
4. Verify the price didn't change

▶ Hint

Exercise 1.3: Multiple Operations in One Transaction

Write a transaction that:

1. Inserts a new user with username 'test_user'
2. Inserts a purchase in user_library for this user
3. Updates the user's total_spent
4. Commits everything together

▶ Hint

Exercise 1.4: Understanding Atomicity

Explain what happens if one operation fails in a transaction. For example:

```
BEGIN;
INSERT INTO users (username, email) VALUES ('john', 'john@example.com');
INSERT INTO users (username, email) VALUES ('john', 'jane@example.com'); -
- Duplicate username!
COMMIT;
```

▶ Hint

Part 2: Savepoints (10 minutes)

Exercise 2.1: Basic Savepoint

Write a transaction that:

1. Starts a transaction
2. Inserts a new publisher 'Test Publisher'
3. Creates a SAVEPOINT called 'after_publisher'
4. Inserts a new developer 'Test Developer'
5. Rolls back to the savepoint
6. Commits the transaction

What data remains in the database?

▶ Hint

Exercise 2.2: Multiple Savepoints

Write a transaction that:

1. Starts a transaction
2. Inserts genre 'Horror'

3. Creates SAVEPOINT 'sp1'
4. Inserts genre 'Mystery'
5. Creates SAVEPOINT 'sp2'
6. Inserts genre 'Thriller'
7. Rolls back to 'sp1'
8. Commits

Which genres are saved to the database?

▶ Hint

Exercise 2.3: Partial Rollback Real-World Scenario

Imagine processing a bulk game purchase:

- Add 3 games to a user's library
- If game 2 fails (already owned), rollback only that game
- Continue with game 3
- Commit successful purchases

Write this using savepoints.

▶ Hint

Part 3: Transaction Isolation Issues (10 minutes)

Read about these common concurrency problems:

Problem Types

1. **Dirty Read**: Reading uncommitted changes from another transaction
2. **Non-Repeatable Read**: Same query returns different results because another transaction modified data
3. **Phantom Read**: Same query returns different number of rows because another transaction inserted/deleted rows
4. **Lost Update**: Two transactions update the same row, one update is lost

---

Exercise 3.1: Identify Dirty Read

Scenario:

```
Transaction A: BEGIN;
Transaction A: UPDATE games SET price = 99.99 WHERE game_id = 1;
Transaction B: BEGIN;
Transaction B: SELECT price FROM games WHERE game_id = 1;  -- Sees 99.99
Transaction A: ROLLBACK;
```

Question: What problem occurs if Transaction B reads the uncommitted price?

▶ Hint

Exercise 3.2: Identify Lost Update

Scenario:

```
Game price is currently $50
Transaction A: BEGIN;
Transaction A: SELECT price FROM games WHERE game_id = 1;  -- Reads $50
Transaction B: BEGIN;
Transaction B: SELECT price FROM games WHERE game_id = 1;  -- Reads $50
Transaction A: UPDATE games SET price = 55 WHERE game_id = 1;
Transaction A: COMMIT;
Transaction B: UPDATE games SET price = 60 WHERE game_id = 1;
Transaction B: COMMIT;
```

Question: What happens to Transaction A's update? What should the final price be?

▶ Hint

Exercise 3.3: Identify Non-Repeatable Read

Scenario:

```
Transaction A: BEGIN;
Transaction A: SELECT price FROM games WHERE game_id = 1;  -- Reads $50
Transaction B: BEGIN;
Transaction B: UPDATE games SET price = 75 WHERE game_id = 1;
Transaction B: COMMIT;
Transaction A: SELECT price FROM games WHERE game_id = 1;  -- Reads $75
Transaction A: COMMIT;
```

Question: Why does Transaction A get different results from the same query?

▶ Hint

Part 4: Isolation Levels (8 minutes)

PostgreSQL supports different isolation levels to prevent concurrency issues:

- `READ UNCOMMITTED`: Lowest isolation (not actually different from READ COMMITTED in PostgreSQL)
- `READ COMMITTED`: Default, prevents dirty reads
- `REPEATABLE READ`: Prevents dirty reads and non-repeatable reads
- `SERIALIZABLE`: Highest isolation, prevents all concurrency issues

Exercise 4.1: Set Isolation Level

Write a transaction that uses REPEATABLE READ isolation level and selects all games.

▶ Hint

Exercise 4.2: Serializable Transaction

Write a transaction that:

1. Uses SERIALIZABLE isolation
2. Checks if a user has enough budget (total_spent < 500)
3. If yes, purchases a game (insert into user_library)
4. Updates total_spent
5. Commits

▶ Hint

Exercise 4.3: Understanding Isolation Levels

Match each isolation level with what it prevents:

a) READ COMMITTED
b) REPEATABLE READ
c) SERIALIZABLE

Problems:

1. Dirty reads
2. Non-repeatable reads
3. Phantom reads
4. All concurrency anomalies

▶ Hint

Part 5: Practical Scenarios (12 minutes)

Scenario 5.1: Purchase Transaction

Write a complete transaction for a game purchase that:

1. Checks if the game exists and gets its price
2. Checks if the user exists
3. Inserts the purchase into user_library
4. Updates the user's total_spent by adding the game price
5. Either commits everything or rollback if any step fails

Include proper error handling logic.

▶ Hint

Scenario 5.2: Review Submission with Concurrency Control

Write a transaction that:

1. Uses REPEATABLE READ isolation
2. Checks if the user owns the game (in user_library)
3. Checks if the user already reviewed this game
4. If checks pass, inserts the review
5. Commits

Why is REPEATABLE READ important here?

▶ Hint

Scenario 5.3: Achievement Unlock

Write a transaction that:

1. Checks if user hasn't already unlocked the achievement
2. Inserts into user_achievements
3. Creates a SAVEPOINT
4. Tries to update a summary table (doesn't exist)
5. Rolls back to savepoint if update fails
6. Still commits the achievement unlock

▶ Hint

Scenario 5.4: Concurrent Game Price Update

Two admins want to update the same game's price. Write a transaction using SERIALIZABLE that:

1. Reads the current price
2. Calculates a 10% increase
3. Updates the price
4. Commits

Explain why this prevents the lost update problem.

▶ Hint

Scenario 5.5: Bulk User Registration

Write a transaction that registers 3 new users:

- User 1: username='alice', email='alice@test.com'
- User 2: username='bob', email='bob@test.com'
- User 3: username='charlie', email='charlie@test.com'

Use savepoints so that if User 2 fails (duplicate), the others still get registered.

▶ Hint

Part 6: Deadlocks (5 minutes)

A **deadlock** occurs when two transactions wait for each other to release locks.

Exercise 6.1: Identify Deadlock

Scenario:

```
Transaction A: BEGIN;
Transaction A: UPDATE games SET price = 50 WHERE game_id = 1;
Transaction B: BEGIN;
Transaction B: UPDATE games SET price = 60 WHERE game_id = 2;
Transaction A: UPDATE games SET price = 70 WHERE game_id = 2;  -- Waits
for B
Transaction B: UPDATE games SET price = 80 WHERE game_id = 1;  -- Waits
for A - DEADLOCK!
```

Question: Why does this cause a deadlock? How can you prevent it?

▶ Hint

Exercise 6.2: Prevent Deadlock

Rewrite the scenario above to prevent deadlock by accessing resources in the same order.

▶ Hint

Challenge Questions

Challenge 1: Banking Transaction

If GameVerse had user wallets, write a transaction to transfer $50 from User A to User B, ensuring neither balance goes negative and the total money in the system stays the same.

Challenge 2: Inventory Management

Write a transaction that:

- Checks game stock (assume a new 'stock' column)
- If stock > 0, sell one copy to a user
- Decrease stock by 1
- Use appropriate isolation level to prevent overselling

Challenge 3: Transaction Log

Research and explain how PostgreSQL's Write-Ahead Logging (WAL) ensures transaction durability. What happens if the database crashes before COMMIT?

Key Concepts Summary

Transaction States

1. **Active**: Transaction is executing
2. **Partially Committed**: Final statement executed, but not committed
3. **Committed**: All changes saved permanently
4. **Failed**: Error occurred, cannot proceed
5. **Aborted**: Rolled back, database restored to state before transaction

When to Use Transactions

 Multiple related operations that must succeed together
 Operations involving money or inventory
 Creating records with relationships
 Preventing race conditions

× Simple single-row SELECT queries
× Read-only operations with no concurrency concerns

Locking Rules

- **Shared Lock**: Multiple transactions can read
- **Exclusive Lock**: Only one transaction can write
- PostgreSQL uses MVCC (Multi-Version Concurrency Control) to reduce locking

---

Best Practices

1. **Keep transactions short**: Lock time should be minimal
2. **Access resources in order**: Prevents deadlocks
3. **Use appropriate isolation levels**: Balance between consistency and performance
4. **Handle errors gracefully**: Always have ROLLBACK logic
5. **Don't hold transactions during user input**: Leads to long locks
6. **Test concurrent scenarios**: Simulate multiple users
7. **Monitor for deadlocks**: Log and analyze patterns

Good luck!